Ansible Avancé : Rôles, gabarits Jinja2 et sécurisation avec Vault

1. Modularisation de l'infrastructure via les Rôles

Le développement d'un playbook monolithique regroupant l'ensemble des tâches d'une infrastructure pose rapidement des problèmes de maintenabilité. Un fichier unique de plusieurs centaines de lignes devient complexe à auditer, difficile à déboguer et impossible à réutiliser sur d'autres projets.

Le concept de **Rôle** permet de découper la configuration en composants autonomes, portables et standardisés (ex: un rôle pour Nginx, un rôle pour PostgreSQL, un rôle de durcissement système).

1.1 Structure standard d'un rôle

La création d'un rôle s'effectue via l'utilitaire en ligne de commande `ansible-galaxy`, qui génère une arborescence de répertoires normalisée.

Commande de génération : `ansible-galaxy role init roles/web_nginx`

Cette commande produit l'arborescence suivante :

- `tasks/main.yml` : Contient la liste des actions à exécuter. C'est le point d'entrée du rôle.
- `handlers/main.yml` : Contient les gestionnaires d'événements (redémarrage de services).
- `defaults/main.yml` : Définit les variables par défaut du rôle (priorité la plus faible).
- `vars/main.yml` : Définit les variables internes du rôle (haute priorité, non destinées à être écrasées).
- `templates/` : Stocke les gabarits de configuration dynamiques (fichiers `.j2`).
- `files/` : Stocke les fichiers statiques à copier tels quels sur la cible.
- `meta/main.yml` : Documente les dépendances du rôle (ex: ce rôle nécessite d'abord l'exécution du rôle de socle).

1.2 Orchestration des rôles dans le playbook principal

Un rôle n'est jamais exécuté directement. Il est appelé par un playbook d'orchestration (généralement nommé `site.yml`), qui associe des rôles à des groupes de l'inventaire.

```yaml
---
```

```
- name: Deploiement des serveurs web
  hosts: web_servers
  become: true
  roles:
    - role: roles/common_baseline
    - role: roles/web_nginx
```

2. Gestion des variables et ordre de précédence

Ansible permet de définir des variables à de multiples emplacements. Pour garantir un comportement prévisible, le moteur applique un ordre de précédence strict. Si une variable est définie avec le même nom à deux endroits différents, l'emplacement ayant la plus haute priorité écrase la valeur précédente.

Les niveaux de précédence les plus courants, du plus faible au plus fort, sont :

1. **defaults/main.yml (Rôle) :** Valeurs par défaut, faciles à écraser.
2. **inventory.ini :** Variables définies au niveau du groupe ou de la machine.
3. **vars/main.yml (Rôle) :** Constantes techniques internes au rôle.
4. **Bloc vars (Playbook) :** Variables définies directement dans le `site.yml`.
5. **Appel de rôle (Playbook) :** Surcharge locale passée lors de l'inclusion du rôle.
6. **Ligne de commande (-e) :** Priorité absolue, surcharge temporaire.

Exemple de surcharge d'une variable lors de l'appel du rôle dans le playbook :

```yaml
- name: Deploiement de l'instance web de test
  hosts: node2
  roles:
    - role: roles/web_nginx
      vars:
        nginx_listen_port: 8080
```

3. Modélisation dynamique avec Jinja2

La distribution de fichiers de configuration statiques via le module `copy` est insuffisante pour gérer un parc hétérogène (adresses IP différentes, noms d'hôtes distincts, ports variables).

Le module `template` déploie des gabarits basés sur le moteur de rendu **Jinja2** (extension `.j2`). Avant d'être transféré sur la cible, le fichier est calculé sur le nœud de contrôle : les variables et les conditions sont évaluées et remplacées par leurs valeurs textuelles.

3.1 Syntaxe Jinja2

L'injection d'une variable s'effectue avec une double accolade : `listen {{ nginx_listen_port }};`

Il est possible d'utiliser les *Facts* (caractéristiques matérielles et réseau collectées automatiquement par Ansible en début d'exécution) : `server_name {{ ansible_hostname }}; bind_address {{ ansible_default_ipv4.address }};`

4. Les déclencheurs conditionnels (Handlers)

Un *handler* est une tâche asynchrone qui ne s'exécute que si elle a été notifiée par une autre tâche ayant modifié le système (statut `changed`).

Il permet d'optimiser les redémarrages de services : si un playbook modifie 4 fichiers de configuration de Nginx, le gestionnaire `Recharger Nginx` sera notifié 4 fois, mais le démon Nginx ne sera rechargé qu'une seule fois, en toute fin de cycle d'exécution.

5. Gestion des secrets avec Ansible Vault

Les mots de passe de bases de données, les clés API et les jetons d'authentification ne doivent pas être stockés en clair dans les fichiers YAML versionnés. **Ansible Vault** permet de chiffrer des fichiers ou des variables à l'aide de l'algorithme AES-256.

5.1 Utilisation en ligne de commande

Création d'un nouveau fichier chiffré : `ansible-vault create vars/vault.yml`

Chiffrement d'un fichier en clair existant : `ansible-vault encrypt vars/vault.yml`

Édition d'un fichier chiffré (déchiffrement en mémoire, puis rechiffrement à la sauvegarde) : `ansible-vault edit vars/vault.yml`

5.2 Automatisation de l'authentification Vault

Pour exécuter un playbook contenant des secrets chiffrés, Ansible requiert le mot de passe maître. Pour éviter une invite interactive en production, le mot de passe est stocké dans un fichier local protégé en lecture stricte (`chmod 0400 ~/.vault_pass`).

Ce fichier est ensuite référencé de manière transparente dans `ansible.cfg` :
`vault_password_file = ~/.vault_pass`

6. Mémo technique opérationnel

Cette section synthétise les modules requis pour la conception des rôles et l'administration système courante.

6.1 Gestion du système et des utilisateurs

Création d'un compte utilisateur avec un répertoire personnel :

```yaml
- name: Creer le compte auditeur
  ansible.builtin.user:
    name: sys-auditor
    shell: /bin/bash
    create_home: true
    state: present
```

Injection d'une clé SSH publique lue depuis un fichier situé sur le nœud de contrôle (Master), via la fonction `lookup` :

```yaml
- name: Deployer la cle SSH
  ansible.posix.authorized_key:
    user: sys-auditor
    state: present
    key: "{{ lookup('file', '~/.ssh/id_auditor.pub') }}"
```

6.2 Gestion des paquets et délai de cache

Installation de paquets multiples et actualisation conditionnelle de la liste des paquets (évite un rafraîchissement à chaque exécution si le cache a moins d'une heure) :

```yaml
- name: Installer les paquets requis
  ansible.builtin.apt:
    name:
      - curl
      - htop
```

```
    state: present
    update_cache: true
    cache_valid_time: 3600
```

6.3 Gestion des fichiers, gabarits et liens symboliques

Déploiement d'un fichier dynamique (Jinja2) et notification au service (Handler) :

```yaml
- name: Deployer la configuration
  ansible.builtin.template:
    src: site.conf.j2
    dest: /etc/nginx/sites-available/custom.conf
    owner: root
    group: root
    mode: '0644'
  notify: Recharger Nginx
```

Création d'un lien symbolique (pour activer un virtualhost) :

```yaml
- name: Activer le site web
  ansible.builtin.file:
    src: /etc/nginx/sites-available/custom.conf
    dest: /etc/nginx/sites-enabled/custom.conf
    state: link
  notify: Recharger Nginx
```

Suppression d'un fichier obsolète ou par défaut :

```yaml
- name: Supprimer le site par defaut
  ansible.builtin.file:
    path: /etc/nginx/sites-enabled/default
    state: absent
  notify: Recharger Nginx
```